

```

        builder: (BuildContext context, AsyncSnapshot snapshot) {
            if (!snapshot.hasData) {
                return Center(child: CircularProgressIndicator());
            }
            return ListView.builder(
                itemCount: snapshot.data.docs.length,
                itemBuilder: (context, index){

                    Map<String,dynamic> document =snapshot.data.docs[index].data() as
Map<String,dynamic>
                    ;

                    return InkWell(
                        onTap: (){
                            Navigator.push(context, MaterialPageRoute(builder:
(builder)=>UpdateSales(
                                document: document,
                                id:snapshot.data.docs[index].id,
                                )));},
                        child: TransCard(
                            amount: document["salesPrice"]==null?0:document["salesPrice"],
                            check: true,
                            totalAmount:
document["salesTotalPrice"]==null?0:document["salesTotalPrice"],
                            QTY: document["salesQTY"]==null?0:document["salesQTY"],
                            unPaid: document["saleskLoan"]==null?0:document["saleskLoan"],
                            itemName: document["salesName"]==null?"No
Data":document["salesName"],
                            nickName: document["salesNickName"]==null?"No
Data":document["salesNickName"],
                            label: "Sold",
                            Time: formattedTime(document["dateTime"]),
                            Date: formattedDate(document["dateTime"])

                        ),

                    );
                });
            },
        );
    }

    String formattedDate(timestamp){
        var
dateFromTimeStamp=DateTime.fromMillisecondsSinceEpoch(timestamp.seconds*1000);
        return DateFormat('dd-MM-yyyy').format(dateFromTimeStamp);
    }
    String formattedTime(timestamp){
        var
dateFromTimeStamp=DateTime.fromMillisecondsSinceEpoch(timestamp.seconds*1000);
        return DateFormat('hh:mm a').format(dateFromTimeStamp);
    }
}

```

Codes For Money Transactions :

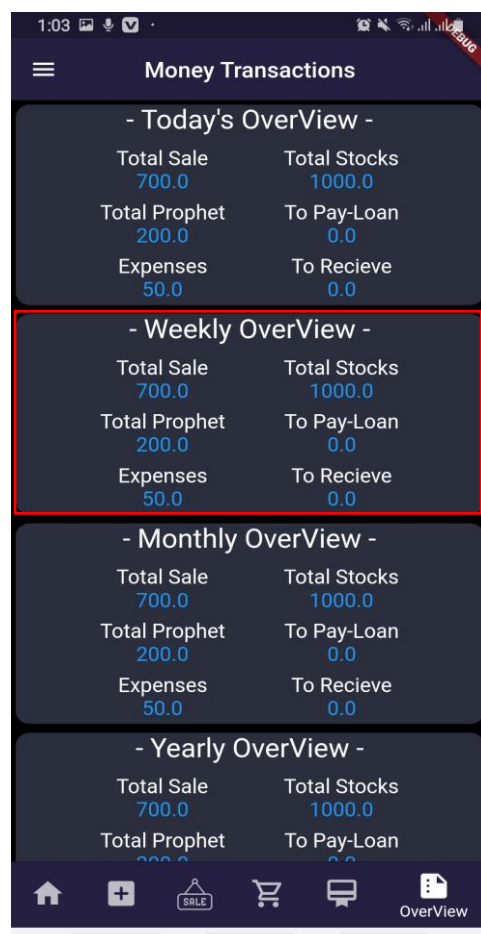
As We See screenshots We noticed that we have five type of transactions on the base of time **Such as** : Todayes Overview , Weekly Overview , Monthly and Yearly .
As a sample we bring weekly overview coz the coding are almost is same .

The following function streamStockweek result is a date time of the last seven days , by using this date time we retrieve weekly Money Transaction.

```
final _streamStockweek=
    FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
        .collection("Transactions").where("dateTime", isLessThan: now,
isGreaterThan: weekly).get();
```

Use Case of the Following Container :

The following container is the Continuation code of Money Transaction Screen and this container is belong to Weekly Overview , we used some Rows And Columns to have good UI table of Results.



Weekly Transactions overview

```
Container(
    width: MediaQuery.of(context).size.width,
    height: 190,
    child: Card(
```

```

shape: RoundedRectangleBorder(
  borderRadius: BorderRadius.circular(12)),
color: Color(0xff2a2e3d),
child: Column(
  children: [
    Text("- Weekly OverView -" ,style: TextStyle(
      fontSize: 24,

      color: Colors.white)),
    SizedBox(height: 10),
    Row(

      mainAxisAlignment:MainAxisAlignment.center,
      children: [

        SizedBox(

          ),
        Column(
          crossAxisAlignment: CrossAxisAlignment.center,
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              'Total Sale',
              style: TextStyle(
                fontSize: 18,

                color: Colors.white)),
          ],
        ),
      ],
    ),
  ],
),

```

Total Sales of Last Week :

As we mention before we have some Rows and Columns inside every Container of Transactions Screen and inside them we have some labels and some values that retrieved from firestore database , the following function is used to retrieve weekly Sales .

```

FutureBuilder(
  future: _streamSaleweek ,
  builder:
    (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {
      sum = 0;

      if (querySnapshot.hasError) {
        return Text("NaN", style: TextStyle(
          fontSize: 18,

          color: Colors.red));
      }

      if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {
        return Text("NaN", style: TextStyle(
          fontSize: 18,

```

```

        color: Colors.red));
    }

    if (querySnapshot.connectionState == ConnectionState.done) {
      querySnapshot.data!.docs.forEach((doc) {
        sum = sum + doc["salesTotalPrice"]; // make sure you create the
variable sumTotal somewhere
      });

      return Text( "${sum}", style: TextStyle(
        fontSize: 18,

        color: Colors.blue));
    }

    return Text("loading", style: TextStyle(
      fontSize: 18,

      color: Colors.red));
  },
),

    SizedBox(
      height: 7,

    ),
    Text(
      'Total Prophet',
      style: TextStyle(
        fontSize: 18,

        color: Colors.white),
    ),
),

```

Total Prophet of the Last Week :

The Following FutureBuilder is for the Result of total Prophet of the last Week .
 We already have the prophet of each selling deal in our database , in this function we just calculate
 The sum of weekly prophets .

```

FutureBuilder(
  future: _streamSaleweek ,
  builder:
    (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {
      sum = 0;

      if (querySnapshot.hasError) {
        return Text("NaN", style: TextStyle(
          fontSize: 18,

```

```

        color: Colors.red));
    }

    if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {
        return Text("NaN", style: TextStyle(
            fontSize: 18,

            color: Colors.red));
    }

    if (querySnapshot.connectionState == ConnectionState.done) {
        querySnapshot.data!.docs.forEach((doc) {
            sum = sum + doc["salesProphet"]; // make sure you create the
variable sumTotal somewhere
        });

        return Text( "${sum}", style: TextStyle(
            fontSize: 18,

            color: Colors.blue));
    }

    return Text("loading", style: TextStyle(
        fontSize: 18,

        color: Colors.red));

    },
),

    SizedBox(
        height: 7,

    ),
    Text(
        'Expenses',
        style: TextStyle(
            fontSize: 18,

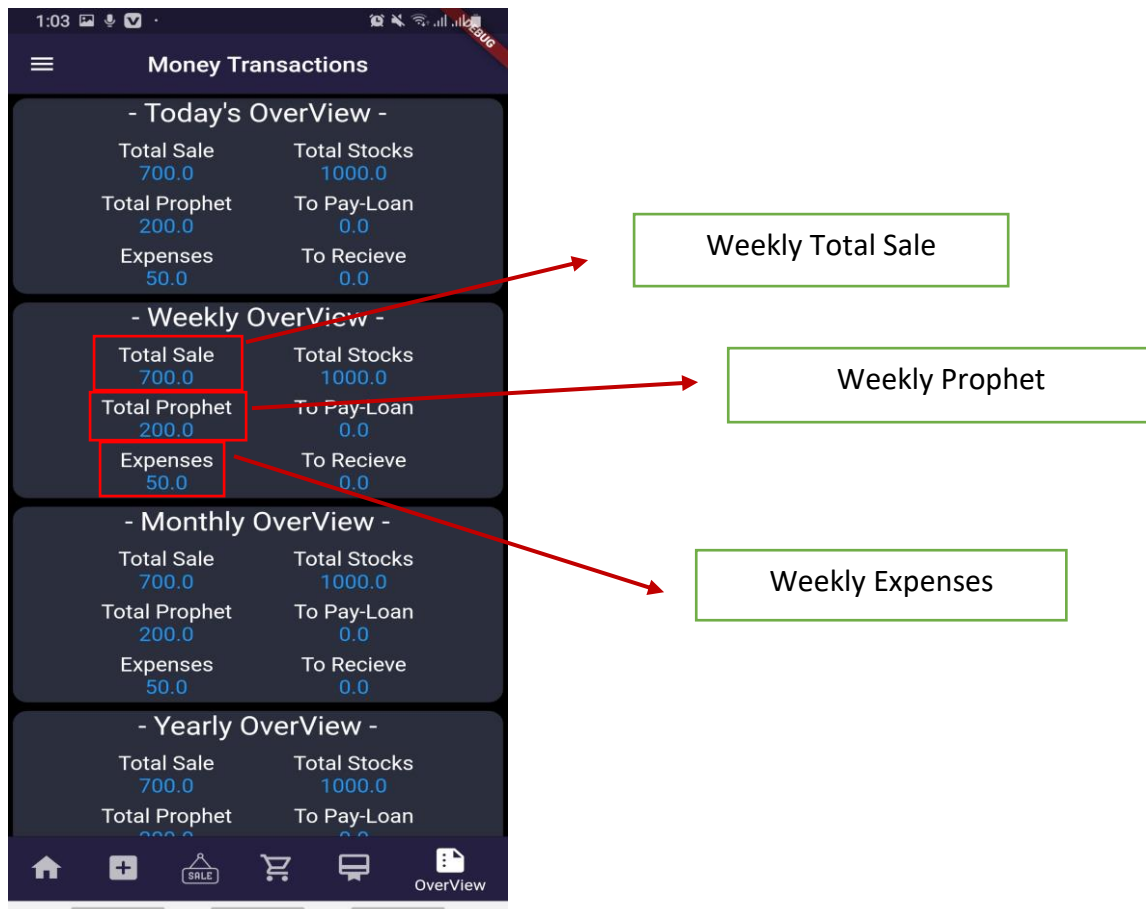
            color: Colors.white),
    ),

```

Expenses of Last Week:

The Following FutureBuilder is suppose to calculate the Sum of Last Week Expenses.

Lets Have a Look to Screenshot What we have done !



```
FutureBuilder(  
  future: _streamExpenseweek,  
  builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {  
    sum = 0;  
  
    if (querySnapshot.hasError) {  
      return Text("NaN", style: TextStyle(  
        fontSize: 18,  
  
        color: Colors.red));  
    }  
  
    if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {  
      return Text("NaN", style: TextStyle(  
        fontSize: 18,  
  
        color: Colors.red));  
    }  
  
    if (querySnapshot.connectionState == ConnectionState.done) {  
      querySnapshot.data!.docs.forEach((doc) {  
        sum = sum + doc["sum"];  
      });  
    }  
  })
```

```

        sum = sum + doc["amount"]; // make sure you create the variable
sumTotal somewhere
    });

    return Text( "${sum}", style: TextStyle(
        fontSize: 18,

        color: Colors.blue));
}

return Text("loading", style: TextStyle(
    fontSize: 18,

    color: Colors.red));

},
),

    ],
),

    SizedBox(
        width: 50,
    ),

    Column(
        crossAxisAlignment: CrossAxisAlignment.center,
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
            Text(
                'Total Stocks',
                style: TextStyle(
                    fontSize: 18,

                    color: Colors.white),
            ),
        ],
    ),

```

Weekly Total Stocks Cost:

The Following FutureBuilder is Supposed to get the last week's all stocks Cost from the firestore and get the sum of all week's Stock Costs.

```

FutureBuilder(
    future: _streamStockweek ,
    builder:
        (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {
        sum = 0;

        if (querySnapshot.hasError) {

```

```

        return Text("NaN", style: TextStyle(
            fontSize: 18,

            color: Colors.red));
    }

    if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {
        return Text("NaN", style: TextStyle(
            fontSize: 18,

            color: Colors.red));
    }

    if (querySnapshot.connectionState == ConnectionState.done) {
        querySnapshot.data!.docs.forEach((doc) {
            sum = sum + doc["stockTotalCost"]; // make sure you create the
variable sumTotal somewhere
        });

        return Text( "${sum}", style: TextStyle(
            fontSize: 18,

            color: Colors.blue));
    }

    return Text("loading", style: TextStyle(
        fontSize: 18,

        color: Colors.red));

    },
),

    SizedBox(
        height: 7,

    ),
    Text(
        'To Pay-Loan',
        style: TextStyle(
            fontSize: 18,

            color: Colors.white),
    ),
),

```

Unpaid Amounts of Last Week :

The Following FutureBuilder will be get the total of unpaid amounts of stock Costs of last Week and will calculate the sum of it .

As we mention before in this section it's all about Money and Money Transactions.

```

FutureBuilder(

```

```

future: _streamStockweek ,
builder:
  (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {
    sum = 0;

    if (querySnapshot.hasError) {
      return Text("NaN", style: TextStyle(
        fontSize: 18,

        color: Colors.red));
    }

    if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {
      return Text("NaN", style: TextStyle(
        fontSize: 18,

        color: Colors.red));
    }

    if (querySnapshot.connectionState == ConnectionState.done) {
      querySnapshot.data!.docs.forEach((doc) {
        sum = sum + doc["stockLoan"]; // make sure you create the variable
sumTotal somewhere
      });

      return Text( "${sum}", style: TextStyle(
        fontSize: 18,

        color: Colors.blue));
    }

    return Text("loading", style: TextStyle(
      fontSize: 18,

      color: Colors.red));
  },
),

    SizedBox(
      height: 7,

    ),
    Text(
      'To Recieve',
      style: TextStyle(
        fontSize: 18,

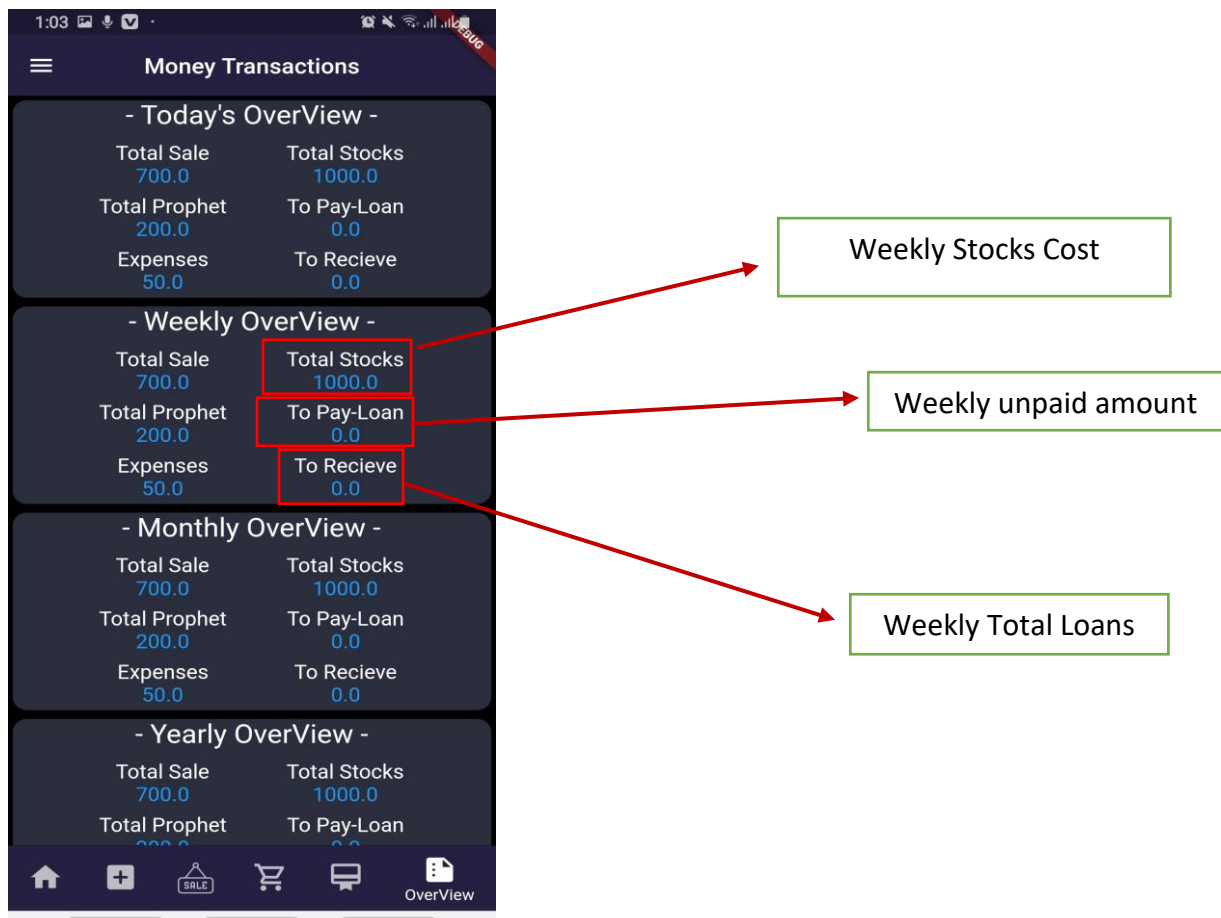
        color: Colors.white),
    ),

```

Loans of Last Week :

The Following FutureBuilder will get the last weeks all unpaid amounts of Customers and will calculate the sum of total loans of last Week.

Let's see to screenshot first !



```
FutureBuilder(  
  future: _streamSaleweek,  
  builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> querySnapshot) {  
    sum = 0;  
  
    if (querySnapshot.hasError) {  
      return Text("NaN", style: TextStyle(  
        fontSize: 18,  
  
        color: Colors.red));  
    }  
  
    if (querySnapshot.hasData && !querySnapshot.data!.docs.isEmpty) {  
      return Text("NaN", style: TextStyle(  
        fontSize: 18,  
  
        color: Colors.red));  
    }  
  })
```


2. Adding Group Accounts will be the main Point that we will work on it, that Any Shop Holder Reach the ability to adding unique account of their stuff and also we will have a chatting Service for stuffs.
3. I have plans to make IMS more user friendly and easy to use

5.3 References:

1. <https://docs.flutter.dev>
2. <https://www.stackoverflow.com>
3. <https://www.geeksforgeeks.org/flutter>
4. <https://www.firebase.google.com>
5. <https://docs.flutter.dev/cookbook>
6. <https://dart.dev>
7. <https://www.javatpoint.com/dart-programming>
8. <https://www.tutorialkart.com/dart>
9. <https://docs.flutter.dev/cookbook>
10. <https://www.atlassian.com/git>

